Quantum Computing II: Shor's Algorithm for Attacking RSA

Paul Joo-Hyun Kim (pauljoohyunkim@gmail.com)

November 2, 2026

Recap of Quantum Computer

Quantum Computer

A computer that uses principles of quantum mechanics for computation.

Misconception

Faster in specific problems. Not faster in general problems.

Warning

This is a very brief description of quantum computing.

Components of Quantum Computing

- States (eg. 0 or 1)
- Qubits (eg. $q = c_0 |0\rangle + c_1 |1\rangle$)
  - Probability of observing state $|i\rangle$ is $|c_i|^2$
- Quantum Gates (eg. Hadamard gate)
  - Must be unitary matrix $H^* H = I$ of dimension $2^n \times 2^n$.
- Tensor Product
  - "Merging" multiple qubits. (Also analogously for applying gates to multiple qubits.)
  - eg: $|0\rangle \otimes |1\rangle = |01\rangle$

Example

Let q be a single qubit such that $q = |0\rangle$. (When you observe you can only see $|0\rangle$ with 100% probability.)

Let $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ be Hadamard gate.

Applying Hadamard gate to q,

$$q \leftarrow Hq = \frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} |1\rangle$$

Now q has 50:50 probability of being observed as $|0\rangle$ or $|1\rangle$. After observing, for example $|1\rangle$, $q \leftarrow |1\rangle$. Any subsequent observation results in $|1\rangle$.

Remark

- There are uncountably infinite number of quantum gates.
- Randomized algorithms are sometimes faster than deterministic algorithms.
- All classical operations can theoretically be implemented with quantum gates.

Shor's Algorithm

Developed by 1994 by Peter Shor.

Goal

Given $N = pq$ (where p, q are prime numbers), find either p or q.

Complexity (Fast Multiplication)

$$O\left((\log N)^2 (\log\log N) \underbrace{(\log\log\log N)}_{\text{Remove if Harvey-Koeven Mult.}}\right)$$

Targets

- RSA
- ECC
  - Shor's algorithm generalizes to discrete logarithm problem.

Algorithm

1. Pick random integer $2 \leq a < N$.
   - Assume $\gcd(a, N) = 1$.
     - Use classical Euclid's algorithm.
     - Otherwise, $\gcd(a, N)$ is a factor of N.
     - By this assumption, a and N are coprime, Euler's theorem says

$$a^{\phi(N)} \equiv 1 \mod N$$

   - Let r be the smallest nonzero integer ("order of modulo group") such that $a^r \equiv 1 \mod N$.
     - Lagrange's theorem guarantees that $r | \phi(N)$
2. Use quantum subroutine to find r.
   - Since $a^r - 1 \equiv 0 \mod N$, we know that $N | \left(a^{r/2} - 1\right)\left(a^{r/2} + 1\right)$.
     - If r is odd, this is not applicable. Start over.
     - Note that $a^{r/2} \not\equiv 1 \mod N$.

$$N \mid \left(a^{r/2} - 1\right)\left(a^{r/2} + 1\right)$$

Algorithm (Cont'd)

3. Compute $g = \gcd\left(N, a^{r/2} + 1\right)$.
   - g is a potential factor.
   - Start over if g is a trivial factor (AKA "1")

Quantum Subroutine

Goal

Given coprime N and $1 < a < N$, find smallest r such that

$$a^r \equiv 1 \mod N$$

Quantum Phase Estimation (QPE)

One needs to be able to solve: given that $|\psi\rangle$ is eigenvector of U, for what θ, is,

$$U|\psi\rangle = e^{2\pi\theta i}|\psi\rangle$$

Quantum Phase Estimation

Goal

$$U|\psi\rangle = e^{2\pi\theta i}|\psi\rangle$$

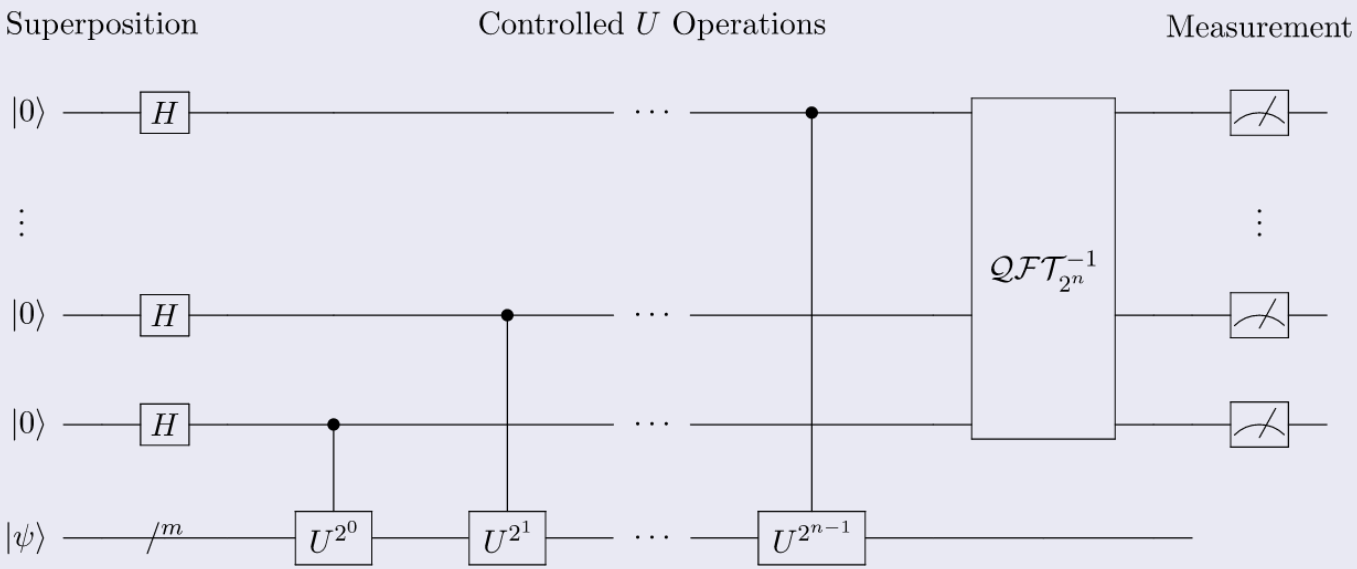
Since θ is a real number between 0 and 1, need a binary representation to extract it out.

Use in Shor's Algorithm

In Shor's algorithm, U describes "modulo (N) multiplication by a". By finding θ, one could find period, then result in "order" r such that $U^r = I$.

Quantum Phase Estimation (QPE) Algorithm

1. Take n additional qubits as "capture register", and define $|\Psi_0\rangle := |0\rangle^{\otimes n} \otimes |\psi\rangle$.

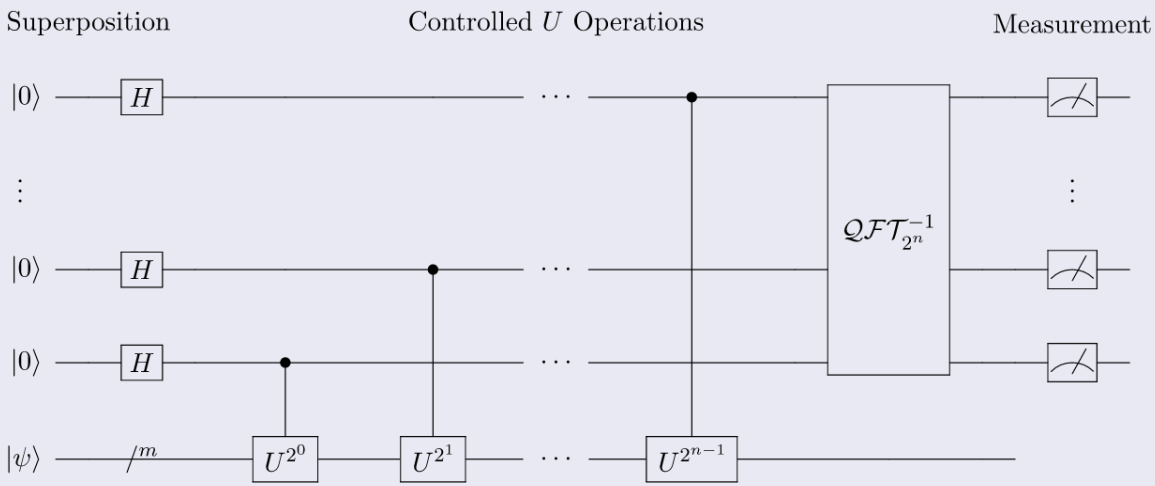


Quantum Phase Estimation (QPE) Algorithm (Cont'd)

2. Apply n-qubit Hadamard gate to the capture register.

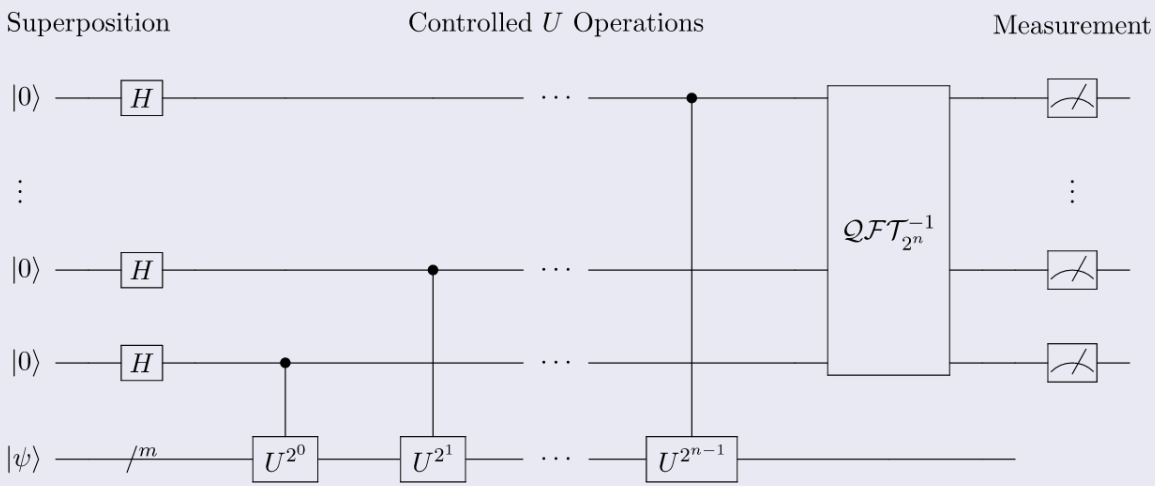
$$|\Psi_1\rangle := \left(H^{\otimes n} \otimes I_m\right)|\Psi_0\rangle$$

$$= \frac{1}{2^{n/2}}\left(|0\rangle + |1\rangle\right)^{\otimes n}|\psi\rangle = \frac{1}{2^{n/2}}\sum_{j=0}^{2^n-1}|j\rangle|\psi\rangle$$



Quantum Phase Estimation (QPE) Algorithm (Cont'd)

3. Apply controlled gate operations $\sum_{k=0}^{2^n-1} |k\rangle\langle k| \otimes U^k$.
   - This applies U^{2^j} only when "capture register" is set to be $|j\rangle$ (in superposition).
   - This "records" information about $e^{2\pi ij}$ in the capture register.



Phase Kickback

After controlled gate operations

$$\sum_{k=0}^{2^n-1} |k\rangle \langle k| \otimes U^k$$

on the capture+target registers, it ends up becoming,

$$\frac{1}{\sqrt{2^n}} \bigotimes_{j=0}^{n-1} \left(|0\rangle + e^{2\pi i \theta \times 2^j} |1\rangle \right) |\psi\rangle$$

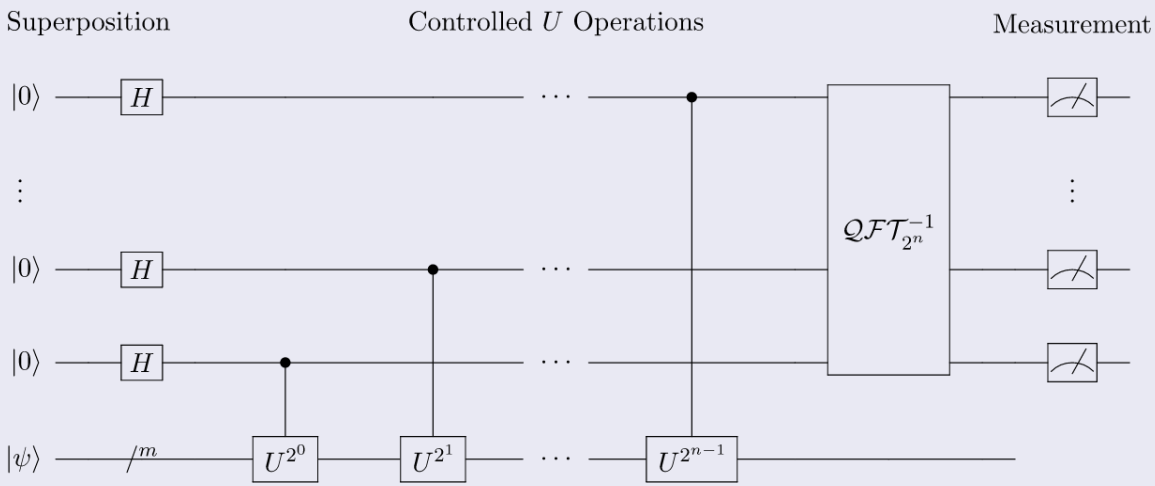
So for $k = 0\text{b}k_{n-1} \cdots k_0$, the total phase is

$$e^{2\pi i \theta (k_{n-1} 2^{n-1})} e^{2\pi i \theta (k_{n-2} 2^{n-2})} \cdots e^{2\pi i \theta k_0 2^0} = e^{2\pi i \theta k}$$

which is the phase for $|k\rangle$.

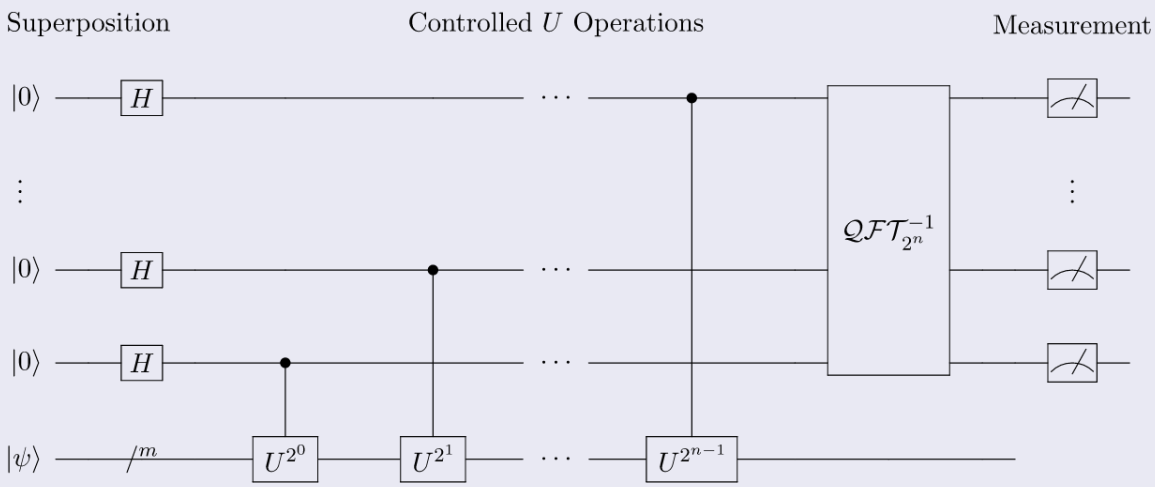
Quantum Phase Estimation (QPE) Algorithm (Cont'd)

4. Apply inverse quantum Fourier transform.
   - Applying iQFT to the capture state $\frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} e^{2\pi i \theta k} |k\rangle$
   - This extracts out the phase information as bitstring in state vector.



Quantum Phase Estimation (QPE) Algorithm (Cont'd)

5. Measure the qubits.
   - Probabilistic, but "very close" to what θ should be.
   - Qubit j stores $|b_j\rangle$ where $\theta \approx \frac{b_1}{2} + \frac{b_2}{4} + \frac{b_3}{8} + \cdots$.



Caveat: QPE for Shor's Algorithm

Note that original quantum phase estimation algorithm **requires** eigenvector, that is some $|\psi\rangle$ such that $U|\psi\rangle = e^{2\pi\theta i}|\psi\rangle$.

Note

Subtlety: Eigenstate is Unknown

For Shor's algorithm, the eigenstate is not known.
But because it is known that $|1\rangle$ is the sum of all eigenstates of U, using QPE will return a phase at random.
This will be j times "order" r of the U, that is, QPE for Shor returns jr such that $U^{jr} = I$ for some j.

Post-Processing: Continued Fraction Algorithm

Measurement yields $\theta \approx \frac{k}{r}$.
A classical algorithm is used to turn the "approximate" phase into fraction and get the denominator r.
This allows the computation of period, and in return, r.
(Hopefully!)

Continued Fraction Algorithm: Idea

Given approximate θ, write

$$\theta = \cfrac{1}{a_0 + \cfrac{1}{a_1 + \cdots}}$$

then, we write the rational approximations,

$$\frac{k}{r} = \frac{1}{a_0}, \frac{1}{a_0 + \frac{1}{a_1}}, \cdots$$

stopping when denominator grows larger than N.
Since all denominators should divide r if approximation is accurate, assume r is LCM of denominators of all rational approximations.

Shor Algorithm Implementation

```
$ pbjk (shor) > time ./bin/shor 15
g was found to be 5 by Shor's algorithm!
5

real    1m47.405s
user    1m46.858s
sys     0m0.539s
```

Implemented with C++ and libsolace.
Note that this was done first in 2001 by IBM with a real 7-qubit quantum computer.

Quantum Subroutine Implementation

```cpp
class ModuloMultiplierGate : public Solace::QuantumGate {
    public:
        ModuloMultiplierGate(const unsigned int n, const unsigned int a, const unsigned int N) : Solace::QuantumGate() {
            if (n == 0 || a <= 1 || a >= N || N == 0 || N > (1U << n)) throw std::invalid_argument("Invalid argument.");

            const auto dim { 1U << n };
            transformer = Solace::SparseQuantumGateTransformer(dim, dim);
            auto& t { std::get<Solace::SparseQuantumGateTransformer>(transformer) };
            for (unsigned int k = 0U; k < dim; k++) {
                if (k < N) {
                    t.coeffRef(k, (a * k) % N) = 1.0;
                } else {
                    t.coeffRef(k, k) = 1.0;
                }
            }
            validate();
        }
};
```

$$U_{k,ak\%N} = 1.0 \qquad (k < N)$$

$$U_{k,k} = 1.0 \qquad k \geq N$$

$$U_{i,j} = 0 \qquad \text{everywhere else}$$

```
static unsigned int quantum_subroutine(const unsigned int a, const unsigned int N, const unsigned n_trials)
    unsigned int n = 1U;
    while (N > (1U << n)) n++;

    ModuloMultiplierGate U { n, a, N };
    std::vector<std::complex<double>> cs (1<<n, 0);
    cs[1] = 1.0;

    // Assertion: estimated_phase is NOT 0 (because this is a trivial one)
    std::vector<unsigned int> rs {};
    for (unsigned int i = 0; i < n_trials; i++) {
        unsigned int estimated_phase { 0U };
        while (estimated_phase == 0) {
            estimated_phase = Solace::Algorithm::quantum_phase_estimation(U, Solace::Qubits(cs), 2*n);
        }

        const auto cfa { continued_fraction_algorithm(estimated_phase, 1<<(2*n)) };
```

cs is $|1\rangle$, sum of all eigenvectors. Note that one uses $2n$ qubits for capture, but this could be reduced at the expense of less accuracy (hence the risk of needing to restart).